

Error Handling in Large Systems

Worked solutions to this chapter's exercises, lab, and review questions.

The exercises in this chapter move error handling out of the realm of message formatting and into the realm of interface design. Each one asks the same underlying question from a different angle: given a failure, what can the caller actually do next, and which layer is responsible for deciding? The warm-up sorts failures into Option, Result, and panic by operational consequence; the code-reading and implementation tasks separate domain language from infrastructure language and replace panic-driven control flow with a small typed contract; and the debugging and design tasks carry those typed contracts across async task boundaries, across a C ABI, and through a full HTTP-to-plugin service path.

The through-line is that a large-system error model survives only when retry policy, translation responsibility, and logging discipline are visible in different places on purpose. The lab is the concrete anchor: a config parser that panics on routine bad input is rewritten so that missing input and malformed input become distinct, matchable variants the caller can branch on.

Exercise solutions

Exercise 1 warm-up comprehension

Choose Option, Result, panic, or process abort from the caller's ability to act

Sort the four events by what the caller can still do, not by how serious they feel. The missing optional request header is ordinary absence: the header may legitimately not be there, and the local code either supplies a default or takes one branch. That is Option<&str> at the point of access. There is no failure to report yet; absence is unremarkable. Only if a later layer decides a missing header is actually a contract violation (an unauthenticated request, say) does that absence get promoted into a typed Result at that boundary.

The database timeout and the invalid state transition are both recoverable, but they belong to different layers. The timeout is an infrastructure failure: the caller can retry, fail over, or dead-letter the work, so it becomes a typed infrastructure error variant such as `InfraError::Timeout` that stays in machinery language and never leaks SQL detail upward. The invalid state transition is a domain failure: the business rule rejected the action, the caller cannot retry the same request and expect a different answer, so it becomes a domain error variant such as `DomainError::InvalidTransition` in business language. Keeping these two in separate enums is what lets the next layer branch correctly: retry on the first, reject on the second.

The impossible-coexistence invariant is the one event that is not a Result at all. If two states the code guarantees are mutually exclusive are both observed, the program's own assumptions are broken, and there is no sensible caller response to return. That is a panic (or, under a strict crash-only policy, a process abort), because continuing past a broken invariant risks producing wrong results rather than merely failing. At the next boundary the recoverable errors are translated, not forwarded verbatim: an `InfraError::Timeout` becomes HTTP 503 with Retry-After, a `DomainError::InvalidTransition` becomes HTTP 409 or 422, an AMQP consumer maps the timeout to a requeue and the domain rejection to a dead-letter, and an FFI boundary collapses both into a status code while the rich detail stays in the Rust-side log.

HOW IT MEETS THE ACCEPTANCE CRITERIA

- Option models the missing header (unremarkable absence), Result with typed variants models the timeout and the transition, and the invariant break is panic/abort rather than a returned error.
- The invalid transition is classified as a domain failure and the timeout as an infrastructure failure, in separate enums.
- The translation step is named per boundary: timeout -> 503/requeue, domain rejection -> 409/dead-letter, both -> status code over FFI, so the same contract does not survive unchanged.

Exercise 2code reading

Separate domain and infrastructure errors in one service path

The billing service returns `anyhow::Result<()>` from deep inside the domain model, which means three failures with three different caller responses arrive as one free-form message chain that nothing downstream can branch on. Split them by layer. The invalid transition is a business-rule failure and belongs in a domain enum, because it speaks the language of the business and tells the caller to reject the request rather than retry it. The SQL timeout and the missing queue channel are machinery failures and belong in an infrastructure or adapter enum, because they speak the language of the plumbing and tell the caller to retry, fail over, or dead-letter. This is exactly the split the chapter's Example 1 demonstrates: a `DomainError` for an empty order, an `InfraError` for a missing store row, and a `ServiceError` that folds both with `#[from]` while preserving which layer failed.

The reason invalid transition must leave the infrastructure bucket is that retry policy depends on it. An infrastructure timeout is often retryable; a rejected business action never is, and retrying it just burns the same request against the same rule. When both are buried in one `anyhow` chain, the worker cannot tell them apart without parsing message text, which is fragile and breaks the moment someone rewords a string. Once the domain layer returns a named variant, the worker writes a real match arm: `retry InfraError::Timeout, dead-letter DomainError::InvalidTransition`, surface anything unrecognized.

`anyhow` does not disappear after the separation; it moves up. It stays useful at the top-level service shell, where the next action is logging, surfacing to a transport, or exiting, and where pattern matching has stopped paying for itself. There you want rich, aggregated context with a source chain, not a giant match. The rule is: typed enums where a downstream layer still branches on the error, `anyhow` where the boundary is already orchestration. The caller actions that typed separation enables are concretely `retry` (on the timeout), `reject` (on the transition), and `dead-letter` (on a poison message) rather than one opaque failed result for all three.

```
use thiserror::Error;

// Domain Language: a business rule said no. Never retryable on its own.
#[derive(Debug, Error)]
enum DomainError {
    #[error("invalid transition from {from} to {to}")]
    InvalidTransition { from: &'static str, to: &'static str },
}

// Machinery Language: the plumbing failed. The caller may retry or dead-letter.
#[derive(Debug, Error)]
enum InfraError {
    #[error("database call timed out after {ms} ms")]
```

```

    SqlTimeout { ms: u64 },
    #[error("queue channel {name} is not available")]
    MissingChannel { name: &'static str },
}

#[derive(Debug, Error)]
enum ServiceError {
    #[error(transparent)]
    Domain(#[from] DomainError),
    #[error(transparent)]
    Infra(#[from] InfraError),
}

// The worker can now branch on real variants instead of message text.
fn classify(err: &ServiceError) -> &'static str {
    match err {
        ServiceError::Infra(InfraError::SqlTimeout { .. }) => "retry",
        ServiceError::Infra(InfraError::MissingChannel { .. }) => "retry",
        ServiceError::Domain(DomainError::InvalidTransition { .. }) => "reject",
    }
}

fn main() {
    let timeout = ServiceError::from(InfraError::SqlTimeout { ms: 5000 });
    let rejected = ServiceError::from(DomainError::InvalidTransition {
        from: "draft",
        to: "paid",
    });
    println!("{}", timeout, classify(&timeout));
    println!("{}", rejected, classify(&rejected));
}

```

Typed domain and infra enums fold into one `ServiceError`, so the worker can match for a retry-versus-reject decision; anyhow stays at the shell above this. Running `main` prints: database call timed out after 5000 ms -> retry invalid transition from draft to paid -> reject

HOW IT MEETS THE ACCEPTANCE CRITERIA

- The invalid transition moves into `DomainError`, out of the infrastructure bucket.
- anyhow is kept only at the top-level shell where matching no longer pays and aggregated context is what is needed.
- The `classify` function names concrete caller actions enabled by typing: retry on the timeout, reject on the transition (and dead-letter on a poison variant).

Exercise 3 implementation

Convert panic-based parsing into typed errors

The original `parse_port` has two `unwrap` calls, and each one panics on a perfectly routine input that a config loader will see in production: `raw.unwrap()` panics when the value is absent, and `parse().unwrap()` panics when the value is present but not a number. Both are recoverable by the caller, so neither should be a panic. The repair is a small typed contract that distinguishes the two cases the caller genuinely needs to tell apart: nothing was provided versus something invalid was provided.

Replace the first `unwrap` with `ok_or`, which converts the `Option<&str>` into a `Result` and attaches the `MissingPort` variant on the `None` branch, then use `?` to short-circuit. Replace the second `unwrap` with `map_err`, which keeps the parsed value on success and rewrites the standard library's `ParseIntError` into the `InvalidPort` variant on failure. The helper returns the parsed `u16` directly and does no logging; logging is a decision for the caller, who has the surrounding context, not for a leaf parser. Keeping the enum to two variants is deliberate: it is small enough to match exhaustively and specific enough that the caller path becomes calm. A missing port might fall back to a default; an invalid port is a real configuration error to surface.

This is the same contract repair most real config loaders need first, and it is exactly what the lab implements. The function no longer panics on the missing or the malformed case, and the return type alone tells the caller which of the two occurred.

```
#[derive(Debug, PartialEq, Eq)]
enum ConfigError {
    MissingPort,
    InvalidPort,
}

fn parse_port(raw: Option<&str>) -> Result<u16, ConfigError> {
    // ok_or turns absence into a typed, matchable error instead of a panic.
    let text = raw.ok_or(ConfigError::MissingPort)?;
    // map_err rewrites the std ParseIntError into our small contract.
    text.parse::<u16>().map_err(|_| ConfigError::InvalidPort)
}

fn main() {
    println!("missing = {:?}", parse_port(None));
    println!("bad = {:?}", parse_port(Some("oops")));
    println!("ok = {:?}", parse_port(Some("8080")));
}
```

`ok_or` handles the missing-input branch and `map_err` handles the parse-failure branch, so the two cases stay distinguishable in the return type with no panic.

HOW IT MEETS THE ACCEPTANCE CRITERIA

- Both `unwrap` calls are gone, so routine missing and invalid input no longer panic.
- `MissingPort` and `InvalidPort` are separate variants, so the caller can tell absence from malformed input from the return type.
- Running the function prints the missing, invalid, and success cases exactly as the lab expects.

Exercise 4debugging or refactoring

Add context to async task propagation

`tokio::spawn` returns a `JoinHandle<Result<T, E>>`, so `.await` on it yields `Result<Result<T, E>, JoinError>`: two stacked failure layers, and the bug is that the inherited code collapses both with a single `.await?` and loses the distinction. The outer layer is the join result, which tells you whether the task finished at all: it returns a `JoinError` if the task panicked or was cancelled. The inner layer is whatever the task body returned: an IO error, a domain rejection, an `anyhow::Error`. A panic inside the task is a different incident from a typed adapter or domain failure, and the unhelpful error text comes from flattening the two so you can no longer tell which happened.

Add context at two distinct points, mirroring the chapter's Example 2. Inside the task body, after the inner `await`, attach operation-level context that names what the work was doing, for example `.context("loading manifest for job")`. Outside, after the join `await`, attach task-level context that names the spawn boundary, for example `.context("manifest task failed")`. Then resolve the two layers with two separate steps rather than one `?`: the chapter's Example 2 uses `.await.context("...task failed")??` for brevity, where the first `?` unwraps the `JoinError` layer and the second unwraps the inner error. The solution below instead expands that into an explicit match on the join result. Both are correct; the `??` form is tighter when you only need success-or-fail, and the match form is clearer here because it lets the cancellation branch be handled separately from a panic and from an ordinary inner error. The context message should name the operation, not repeat the same low-level noun: "loading manifest" is diagnosable, a second "not found" line is not. Useful fields are the job ID, queue name, attempt number, and the resource name being touched, because those are what an operator needs to reconstruct the incident.

Retryability is a property of the inner error, not of the join layer, and it should be recorded on the error type rather than inferred from message text. A transient inner failure (a timeout, a temporarily unavailable dependency) is retryable, so the caller can requeue with backoff; a domain rejection that came back through the task is terminal and retrying it just repeats the same answer. Expose that as a method such as `is_retryable` on the inner error so the worker reads one flag instead of substring-matching the formatted context. A join panic is its own case: it usually signals a bug, so it is not silently retried.

Cancellation should be modeled distinctly from ordinary inner failure whenever the system can shut a task down deliberately. `JoinError` exposes `is_cancelled()` and `is_panic()`; a task aborted during graceful shutdown is not a failure of the work and must not be reported as a generic timeout or a generic error string, because that would page someone for a clean shutdown. Branch on the `JoinError`: a cancel during shutdown is expected and logged at low severity, a panic is a bug that should surface loudly, and only the inner error carries the real domain or infrastructure failure.

```
use anyhow::{Context, Result};

async fn do_work(job: u64) -> Result<u64> {
    if job == 0 {
        anyhow::bail!("empty job payload");
    }
    Ok(job * 2)
}

async fn run(job: u64, queue: &str, attempt: u32) -> Result<u64> {
    let handle = tokio::spawn(async move {
        // Operation-layer context: names the work, not a repeated noun.
        do_work(job)
            .await
            .with_context(|| format!("processing job {job}"))
    });

    match handle.await {
        // Inner Result resolved here: '?' carries the operation error up,
        // with task-layer context naming the spawn boundary.
        Ok(inner) => inner.with_context(|| {
            format!("job {job} task on queue {queue} (attempt {attempt}) failed")
        }),
        Err(join_err) if join_err.is_cancelled() => {
            // Cancellation is not an ordinary failure: do not collapse to a timeout.

```

```

        anyhow::bail!("job {job} cancelled during shutdown")
    }
    Err(join_err) => Err(join_err)
        .with_context(|| format!("job {job} task panicked on queue {queue}")),
    }
}

```

The join layer (`handle.await`) and the inner operation layer (`do_work`) are resolved separately, each with its own context, and cancellation is branched out before generic failure handling. This is a pattern fragment, not a standalone program: it needs a tokio runtime, so call `run` from inside a `#[tokio::main]` `async fn main` (for example, `run(1, "orders", 1).await`) to compile and run it.

HOW IT MEETS THE ACCEPTANCE CRITERIA

- The join layer and the inner error layer are resolved in separate match arms rather than one collapsed `?`.
- Operation-layer context ("processing job N") and task-layer context ("...task on queue... failed") are added at both points.
- `is_cancelled()` is branched out so a shutdown-cancel is reported distinctly and never as a generic timeout.

Exercise 5debugging or refactoring

Translate a Rust error contract across FFI without leaking internals

An extern "C" function that returns `anyhow::Result<u64>` is unsound at the boundary, not merely inelegant. A C ABI cannot carry a Rust enum, an `anyhow::Error`, or the stack unwinding a panic implies; letting any of those cross the boundary is undefined behavior. The other runtime has no way to safely consume an internal `anyhow` chain as if it were a stable public type. So the boundary must become a deliberate narrowing: the rich error stays on the Rust side, and the foreign side sees a small, boring, ABI-safe contract.

Replace `anyhow::Result<u64>` with an explicit status code plus an out parameter for the result. The public types are an `i32` (or a `repr(C)` enum of status codes) for the outcome and a `*mut u64` for the value written on success. Distinguish the foreign-visible failure classes explicitly: a null or zero-length input is one status (for example `STATUS_NULL_INPUT = 1`), invalid UTF-8 in the payload is another (`STATUS_INVALID_UTF8 = 2`), and any internal computation failure is a third generic status (`STATUS_INTERNAL = 3`). Two or three classes are usually enough; the foreign caller only needs to branch on what it can act on. The richer detail, which row failed, which dependency timed out, the full source chain, stays inside a Rust-side log or trace and never escapes over the ABI, because the foreign runtime cannot use it and exposing it pins your internals as a public contract. The whole function body is wrapped so panics cannot unwind across the boundary: `std::panic::catch_unwind` converts any internal panic into the generic internal status.

Test the status boundary with a tiny native harness, a few lines of C that call `run` with a null pointer, with deliberately invalid UTF-8 bytes, and with a valid payload, asserting on the returned status code and the out value each time. That harness verifies the contract the other runtime actually depends on, independently of any Rust-side richness. This is deliberately boring, and the boredom is the feature: a stable, narrow ABI is what lets the foreign side build against the boundary with confidence.

```

use std::panic::catch_unwind;
use std::slice;

#[repr(i32)]
pub enum Status {
    Ok = 0,
    NullInput = 1,
    InvalidUtf8 = 2,
    Internal = 3,
}

/// ABI-safe boundary: status code out, result via out-parameter.
/// Rich Rust errors stay behind this wrapper and are only logged.
#[no_mangle]
pub extern "C" fn run(ptr: *const u8, len: usize, out: *mut u64) -> i32 {
    if ptr.is_null() || out.is_null() {
        return Status::NullInput as i32;
    }

    // catch_unwind keeps a panic from unwinding across the C ABI (UB).
    let result = catch_unwind(|| {
        let bytes = unsafe { slice::from_raw_parts(ptr, len) };
        let text = match std::str::from_utf8(bytes) {
            Ok(t) => t,
            Err(_) => return Err(Status::InvalidUtf8),
        };
        // Real work lives here and may produce a rich internal error,
        // which we log on the Rust side rather than export over the ABI.
        score(text).map_err(|err| {
            eprintln!("internal failure: {err:#}");
            Status::Internal
        })
    });

    match result {
        Ok(Ok(value)) => {
            unsafe { *out = value };
            Status::Ok as i32
        }
        Ok(Err(status)) => status as i32,
        Err(_) => Status::Internal as i32, // a panic was caught
    }
}

fn score(text: &str) -> anyhow::Result<u64> {
    Ok(text.len() as u64) // stand-in for the rich internal path
}

```

anyhow::Result is replaced by an i32 status plus a *mut u64 out-parameter; null, invalid UTF-8, and internal failure are distinct statuses, and catch_unwind stops panics from crossing the ABI.

HOW IT MEETS THE ACCEPTANCE CRITERIA

- The boundary is translated into a status code plus an out-parameter instead of anyhow::Result.
- Null input, invalid UTF-8, and internal failure are three explicit foreign-visible status classes.

- The rich anyhow detail is logged on the Rust side behind the wrapper and never exported as the public ABI, and a native harness calling run with null, bad UTF-8, and a valid payload checks the status contract.

Exercise 6 design or production scenario

Design one error contract for HTTP, queue workers, and a native plugin

Design three layered error families and one explicit policy for retry, logging, and translation, so another team can operate the failure modes without reading the implementation. The domain layer owns business-rule failures: an empty order, an invalid transition, an exhausted quota. These are `DomainError` variants in business language, never mentioning SQL or TCP, and they are terminal because retrying the same request against the same rule changes nothing. The infrastructure layer owns machinery failures: a database timeout, a missing queue channel, a plugin call that did not respond. These are `InfraError` variants in machinery language, and most are retryable. The native risk plugin sits behind the ABI boundary from Exercise 5 and surfaces only a status code, which the adapter maps into an `InfraError` variant (`PluginUnavailable`, `PluginRejected`) so the rest of the system never sees raw ABI codes. At the top, one `ServiceError` folds domain and infrastructure with `#[from]` and is what the HTTP handler and the worker both translate from.

Make retryable-versus-terminal an explicit property, not a guess from message text. The cleanest place is a method on the error, `fn is_retryable(&self) -> bool`, that returns true for infrastructure timeouts and plugin-unavailable, false for every domain rejection and for plugin-rejected. The HTTP handler reads it to choose 503 with `Retry-After` versus 409/422; the queue worker reads it to choose `requeue-with-backoff` versus `dead-letter`. The policy lives in one place, on the type, so HTTP and the worker cannot drift apart.

Separate the boundary that logs from the boundary that only translates, or the incident channel fills with the same failure reworded five times. The rule: log once at the layer where the failure first becomes operationally meaningful, usually the adapter that talks to the failing dependency, because that layer has the resource name and the attempt. Every layer above it translates and returns without re-logging. So the database adapter logs the timeout once with full context; the worker translates it into a `requeue` decision and does not log it again; the HTTP boundary translates a propagated `ServiceError` into a status code and does not log it again. Finally, every failure path carries the same structured fields so the incident is reconstructable: request ID, task ID, attempt number, queue name, and the plugin call name. With those on the log line, an operator reconstructs the whole charge from one entry instead of correlating five partial ones.

```
use thiserror::Error;

#[derive(Debug, Error)]
enum DomainError {
    #[error("order {id} has no lines")]
    EmptyOrder { id: &'static str },
    #[error("risk plugin rejected order {id}")]
    RiskRejected { id: &'static str },
}

#[derive(Debug, Error)]
enum InfraError {
    #[error("database timed out after {ms} ms")]
```



```

    DbTimeout { ms: u64 },
    #[error("risk plugin unavailable")]
    PluginUnavailable,
}

#[derive(Debug, Error)]
enum ServiceError {
    #[error(transparent)]
    Domain(#[from] DomainError),
    #[error(transparent)]
    Infra(#[from] InfraError),
}

impl ServiceError {
    // Retry policy lives on the type, in one place, not in message text.
    fn is_retryable(&self) -> bool {
        matches!(
            self,
            ServiceError::Infra(InfraError::DbTimeout { .. })
            | ServiceError::Infra(InfraError::PluginUnavailable)
        )
    }

    // HTTP boundary: translate only, do not log here.
    fn http_status(&self) -> u16 {
        if self.is_retryable() { 503 } else { 422 }
    }

    // Worker boundary: translate only, do not log here.
    fn worker_action(&self) -> &'static str {
        if self.is_retryable() { "requeue" } else { "dead-letter" }
    }
}

fn main() {
    let timeout = ServiceError::from(InfraError::DbTimeout { ms: 5000 });
    let rejected = ServiceError::from(DomainError::RiskRejected { id: "ord-9" });
    // Fields like request_id/task_id/attempt/queue/plugin_call belong on the
    // single log line emitted by the adapter that first saw the failure.
    println!("timeout: http={} worker={}", timeout.http_status(), timeout.worker_action());
    println!("rejected: http={} worker={}", rejected.http_status(), rejected.worker_action());
}

```

DomainError, InfraError, and a folded ServiceError carry an `is_retryable` method; `http_status` and `worker_action` translate without logging, leaving the one log line to the adapter that first saw the failure.

HOW IT MEETS THE ACCEPTANCE CRITERIA

- At least one domain error, one infrastructure error, and a translated ServiceError (with HTTP/worker transports) are defined.
- `is_retryable` makes retryable-versus-terminal an explicit method on the type rather than a message substring.
- The adapter that first sees the failure logs once; `http_status` and `worker_action` only translate, avoiding duplicate noise.
- Observability fields named for every path: request ID, task ID, attempt, queue, and plugin call name.

Lab solution

The original code panics twice on routine input: `raw.unwrap()` panics when the port value is absent, and `text.parse().unwrap()` panics when the value is present but not numeric. Both are recoverable by the caller, so both become typed errors instead of panics. The fix swaps each `unwrap` for an explicit branch that names the failure.

`raw.ok_or(ConfigError::MissingPort)?` converts `Option<&str>` into `Result<&str, ConfigError>`, attaching the `MissingPort` variant on `None`, and the `?` operator returns it immediately. On the present path, `text.parse::<u16>()` yields `Result<u16, ParseIntError>`, and `map_err(|_| ConfigError::InvalidPort)` rewrites the standard-library parse error into the `InvalidPort` variant while leaving a successful parse untouched. The helper does no logging and returns the parsed value directly, leaving any logging or fallback decision to the caller that has the context.

The three calls in `main` therefore exercise the three distinct paths. `parse_port(None)` takes the `ok_or` branch and prints `Err(MissingPort)`; `parse_port(Some("oops"))` parses and fails, taking the `map_err` branch to print `Err(InvalidPort)`; `parse_port(Some("8080"))` parses successfully and prints `Ok(8080)`. Missing and malformed input are now distinguishable from the return type alone, which is the whole point of the typed contract.

Example 41-S. `typed_error_lab.rs` — *Reference solution*

```
#[derive(Debug, PartialEq, Eq)]
enum ConfigError {
    MissingPort,
    InvalidPort,
}

fn parse_port(raw: Option<&str>) -> Result<u16, ConfigError> {
    // ok_or promotes absence into a typed, matchable error; '?' short-circuits.
    let text = raw.ok_or(ConfigError::MissingPort)?;
    // map_err rewrites the std ParseIntError into our small contract.
    text.parse::<u16>().map_err(|_| ConfigError::InvalidPort)
}

fn main() {
    println!("missing = {:?}", parse_port(None));
    println!("bad = {:?}", parse_port(Some("oops")));
    println!("ok = {:?}", parse_port(Some("8080")));
}
```

OUTPUT

```
missing = Err(MissingPort)
bad = Err(InvalidPort)
ok = Ok(8080)
```

Review question answers

When is `Option` enough, and when should the boundary promote absence into `Result`?

Option is enough when absence is unremarkable and the caller only needs one local branch: supply a default, skip the step, or fall through. You should promote that absence into a typed Result at the boundary where the absence actually becomes a contract violation the caller must classify and handle, for example a missing field that means an invalid request rather than a benign default. The rule is to keep Option at the point of lookup and convert to a named error (often with `ok_or`) exactly where a downstream layer needs to branch on why the value was missing.

Why are domain errors and infrastructure errors worth keeping distinct even if both eventually become one HTTP response or one queue outcome?

They imply different caller actions, and that decision happens above the layer that produced the error, before any HTTP or queue collapse. An infrastructure failure such as a timeout is usually retryable; a domain rejection such as an invalid transition is terminal, and retrying it just repeats the same rejection. If both live in one error bag, the worker cannot tell retry from reject without parsing message text, which is fragile. Keeping them in separate enums lets the next layer match on real variants and choose retry, reject, or dead-letter honestly, even though the final boundary may still fold them into a single status.

What is the practical split between `thiserror` and `anyhow` in a large Rust system?

They answer different questions, and most systems use both. Reach for `thiserror` to define named error enums with named variants when a downstream layer still needs to branch on the error; the macro generates the boilerplate for Display, the source chain, and `#[from]` conversions. Reach for `anyhow` when the boundary is already top-level orchestration, where the next action is logging, surfacing, or exiting, and where exhaustive matching has stopped paying for itself; there you want rich aggregated context rather than a large match. The deciding factor is simply whether anyone downstream still branches on the variant.

Why does a spawned task usually create a two-layer error surface rather than one?

`tokio::spawn` returns a `JoinHandle<Result<T, E>>`, so awaiting it produces `Result<Result<T, E>, JoinError>`: two stacked layers. The outer `JoinError` layer reports whether the task ran to completion at all, returning an error if the task panicked or was cancelled. The inner `Result` is whatever the task body returned, such as an IO error or a domain rejection. Collapsing both with a single `?` loses the distinction between a task that crashed and a task that finished but returned a typed failure, which are different incidents that deserve different handling, so you resolve the two layers deliberately.

Why should a boundary usually log once rather than at every layer that propagates the same error upward?

Logging the same failure at every layer turns the incident channel into a hall of mirrors, where one failure appears five times slightly reworded and an operator cannot tell whether that is one incident or five. The discipline is to log once at the single layer where the failure first becomes operationally meaningful, usually the adapter closest to the failing dependency, which has the resource name and attempt count in hand. Every layer above it translates and returns the error without re-logging, so each failure produces exactly one rich, reconstructable log line.